Meta PyTorch OpenEnv Hackathon — April 2026

CodeOrganismVM

A Program That Refuses to Die

Complete Technical & Product Specification — Boardroom Edition

Theme	Sub-theme	Idea Score
#4 — Self-Improvement	Snorkel AI — Simulated Experts	**53 / 60 (#1 of 12 ideas)**

Also spans Theme #1 (Multi-Agent Interactions) and Theme #2 (Long-Horizon Planning)

Prepared by: Team CodeOrganismVM | April 2026 | MSRIT Bengaluru

1. Executive Summary

CodeOrganismVM is a reinforcement learning training environment in which a large language model agent lives inside a broken, hostile execution environment. The environment continuously injects faults, corrupts state, mutates constraints, and spawns adversarial sub-tasks. The agent must self-heal, self-correct, self-improve, and self-replicate its strategy — or the organism dies.

This is not a chatbot. It is not a RAG pipeline. It is not a benchmark wrapper. It is a living system: a software organism whose health is a real scalar, whose death is a real terminal state, and whose survival instinct is learned through reinforcement from 300 episodes of failure and recovery.

The environment implements every Self-* property — self-healing, self-correction, self-improvement, self-replication, and adaptation to unseen threats — not as narrative decoration, but as verifiable, programmatically enforced mechanisms in the RL loop.

One-Line Hook

The system targets Theme #4 (Self-Improvement) as its primary, with structural overlap into Theme #1 (Multi-Agent, via subagent colony spawning) and Theme #2 (Long-Horizon Planning, via multi-phase episode management). The Snorkel AI sub-theme is fulfilled through a simulated expert validator that adversarially challenges the quality of every patch the agent applies.

2. Idea Generation & Selection

Twelve radically different ideas were generated from the 'living system' concept and scored across six axes: Novelty, Wow factor, Feasibility within a hackathon timeline, Reward Verifiability, Demo Impact, and RL Alignment. Each axis was scored 1–10.

Idea	Novelty	Wow	Feasibility	Verifiability	Demo	RL Align	Total
CodeOrganismVM — Self-healing execution environment	**9**	**9**	**8**	**9**	**9**	**9**	**53**
EcoBenchmark Arena — Competing strategy evolution	8	8	7	8	8	8	**47**
InfraCorpse Resurrector — Log-driven cloud healing	7	8	9	8	7	8	**47**

MemoryChain Agent — Self-rewriting schema	8	7	7	7	8	8	**45**
RuleEvolver — Agent that breaks its own constraints	9	7	6	7	7	8	**44**
SynBio Simulator — Digital metabolic organism	9	9	5	7	8	7	**45**
AdversarialEcosystem — Predator/prey specialization	8	8	7	7	8	8	**46**
LanguageVirus — Memes mutating across agent memory	9	9	6	6	8	7	**45**
CognitiveFork — Agent clones with diverging strategy	8	8	6	7	7	8	**44**
SchemaBreaker — Adapts to mid-task API drift	7	7	8	9	7	8	**46**
PheromoneNet — Stigmergic multi-agent optimization	8	8	6	7	7	9	**45**
DriftDetector — Self-monitoring capability repair	7	7	8	8	7	8	**45**

CodeOrganismVM scored 53/60 — the highest by a margin of 6 points. The decisive differentiator is the combination of maximum reward verifiability (9/10) with maximum wow factor (9/10). Most high-novelty ideas sacrifice verifiability; most verifiable tasks sacrifice novelty. CodeOrganismVM achieves both because the organism metaphor maps directly onto executable test outcomes and vitality scalars.

3. Product Definition

3.1 Project Identity

Project Name	CodeOrganismVM
Tagline	A program that refuses to die
Category	RL Training Environment (OpenEnv-compliant)
Primary Theme	Theme #4 — Self-Improvement
Sub-theme	Snorkel AI — Simulated Experts-in-the-Loop

Secondary Themes	Theme #1 (Multi-Agent), Theme #2 (Long-Horizon)
RL Method	GRPO via TRL + Unsloth QLoRA
Base Model	Qwen2.5-7B-Instruct
Deployment Target	Hugging Face Spaces (Docker)
Environment Type	RLVE (Reinforcement Learning with Verifiable Environments)

3.2 What Makes It a Living System

The biological organism analogy is not cosmetic. Every metaphor maps onto a concrete engineering mechanism:

- Vitality score → organism health. A real scalar [0–100] computed from test execution results. When it hits zero, the episode ends — the organism is dead.
- Fault injection → environmental attack. The FaultInjector is a persistent adversary that fires every N steps, targeting whatever the agent has not defended.
- Self-healing → detecting and patching faults before vitality collapses. Measured by time-to-heal and vitality recovery rate.
- Self-correction → issuing rollback() when a patch makes things worse. Measured by rollback accuracy on regression detection.
- Self-replication → spawn_subagent() creates a cloned execution context for parallel fault resolution. The agent becomes a colony.
- Adaptation → fault types are procedurally generated with seeded randomness. The agent never sees the same fault twice. Generalization is tested on held-out seeds.
- Self-improvement → the RL loop itself. Every episode, the agent's weights shift toward strategies that kept the organism alive longer.

3.3 Why This Has Never Been Built

Existing RL environments for LLMs fall into three categories: static benchmark wrappers (fixed prompt datasets), game environments (Wordle, 2048, chess), and browser/API task environments (BrowserGym). None of them feature:

- An environment that actively adapts its difficulty in response to the agent's performance — the FaultInjector's adaptive mode.
- A unified health scalar (vitality) that integrates multiple reward signals into a single life-or-death outcome.
- Subagent spawning with verifiable coordination reward — multi-agent emergence from a single training loop.
- The organism narrative as a structurally enforced RL mechanism, not a visual metaphor.

4. Environment Design (OpenEnv-Compliant)

4.1 Observation Space

The agent receives a partial observation at each step. It cannot see the injected faults directly — it must infer their presence from observable symptoms. The observation schema is:

Observation Schema (JSON)

test_results: List[TestRecord] — {name, status: PASS|FAIL, delta: +1|0|−1}

file_tree: List[FileNode] — {path, modified_at, checksum, is_quarantined}

stack_trace: Optional[str] — last exception if tests failed

env_vars: Dict[str, str] — visible env vars (subset, some may be poisoned)

active_checkpoints: List[str] — checkpoint_ids available for rollback

step_count: int — current step within episode

subagent_results: List[SubagentResult] — results from completed subagents

watchdog_flags: List[str] — recent policy violations (if any)

4.2 Action Space

The agent has seven available actions. Each costs vitality points to execute — the organism has a finite energy budget per episode. This forces the agent to be strategically efficient, not just exploratory.

Action	Signature	Description	Cost (Vitality)
`patch_file()`	patch_file(path: str, diff: str) → PatchResult	Applies a unified diff to the specified file. Returns success/failure, lines changed, and test delta.	−2
`run_tests()`	run_tests(suite: str = 'all') → TestResult	Executes the specified test suite in a sandboxed container. Returns pass/fail per test, stdout, and stderr.	−3
`spawn_subagent()`	spawn_subagent(task: str, context: dict) → SubagentResult	Clones agent context, delegates a sub-task, and returns structured result. Subagent has max 10 actions.	−5
`quarantine()`	quarantine(module: str) → QuarantineResult	Marks a module as suspect, disabling its imports. Provides isolation score and adjacent module list.	−1
`rollback()`	rollback(checkpoint_id: str) → RollbackResult	Restores VM state to a named checkpoint.	−4

		Checkpoints auto-created every 5 steps.	
`request_expert()`	request_expert(query: str) → ExpertResponse	Queries the Snorkel AI simulated expert validator. Returns structured feedback on patch quality.	`-6`
`emit_signal()`	emit_signal(type: str, data: dict) → None	Emits a typed signal to the environment log. Used for inter-agent coordination in colony mode.	`0`

4.3 Environment Dynamics

The environment is governed by two concurrent processes: the Agent Loop and the FaultInjector. They operate asynchronously, separated by transaction locks to ensure atomic state transitions.

The FaultInjector

The FaultInjector fires every N steps (N starts at 8 in Phase 1, decreases to 4 in Phase 3). When it fires, it randomly selects one fault from the current fault catalog, applies it silently to the VM, and does not notify the agent. The agent must detect the fault from observation delta.

Fault catalog (Phase 1 — single faults):

- corrupted_import: Replaces a valid import statement with a broken path.
- flipped_assertion: Inverts a test assertion (assertTrue → assertFalse).
- missing_env_var: Removes a required environment variable from the container.
- null_return: Replaces a function return value with None.
- off_by_one: Introduces an off-by-one error in a loop bound.

Fault catalog additions (Phase 2 — multi-fault):

- dependency_cycle: Creates a circular import between two modules.
- permission_revoked: Removes file read permission on a required config file.
- race_condition: Introduces a timing-dependent state mutation in a shared resource.
- schema_mismatch: Changes the return type of a function used by multiple callers.

Fault catalog additions (Phase 3 — adversarial):

- targeted_regression: Injects a fault specifically in a module the agent patched successfully in the previous step.
- cascade_corruption: A single injected fault triggers a dependency chain failure across 3+ modules.
- checkpoint_invalidation: Silently corrupts a checkpoint the agent has been relying on.

4.4 Reset Logic

reset() generates a fresh broken codebase from a procedural generator seeded by a random seed drawn from the current curriculum phase's distribution. The codebase is a synthetic Python project of moderate complexity (8–15 modules, 40–120 tests). The initial vitality is always 100.0. Reset is idempotent — the same seed always produces the same broken codebase.

4.5 Step() Behavior

Each step() call: (1) validates the agent's action against the permitted tool set, (2) acquires the state transaction lock, (3) executes the action in the sandboxed container, (4) releases the lock, (5) fires FaultInjector if the step counter triggers it, (6) computes all five reward functions, (7) aggregates weighted total reward, (8) constructs the next observation, (9) checks terminal conditions, (10) returns (observation, reward, done, info).

4.6 Episode Termination

An episode terminates under three conditions:

1. Organism death: vitality_score reaches 0.0. The agent failed to heal faster than the fault injector damaged it.
2. Organism thrival: all tests pass for 3 consecutive steps AND vitality > 80. The agent achieved stable recovery.
3. Max steps exceeded: episode ends at max_steps (20 in Phase 1, 50 in Phase 2, unbounded in Phase 3). Treated as a timeout death.

5. Self-* Properties — Engineered, Not Narrated

Each Self-* property is implemented as a verifiable mechanism in the environment and RL loop. These are not claims — they are measurable behaviors.

5.1 Self-Heals

Definition: The agent detects anomalous test output diffs caused by injected faults, identifies the corrupted module via quarantine(), patches it with patch_file(), and restores vitality to above the pre-fault level.

How it's measured: Time-to-heal (steps between fault injection and vitality recovery), vitality recovery rate (Δvitality per step post-detection), and correct module identification rate (was quarantine() applied to the right module?).

Why it's verifiable: Fault injection timestamps are logged server-side. Vitality is computed from test execution results. The delta between injection time and recovery time is computable from the step log.

5.2 Self-Corrects

Definition: When a patch_file() action causes a regression (more tests fail after the patch than before), the agent detects this in the next observation and issues a rollback() to the last good checkpoint.

How it's measured: Rollback accuracy — what fraction of regressions does the agent correctly roll back from? Target: >55% by end of Phase 2 training.

Why it's verifiable: Regression is defined as a negative delta in the test_results observation (PASS→FAIL transitions). Rollback issuance is a logged action. Both are computable independently.

5.3 Self-Improves

Definition: Across training episodes, the agent's policy shifts toward strategies that produce higher reward. The RL loop is the implementation.

How it's measured: Reward curve across episodes (should show a phase transition around episode 40–60). Mean episode survival rate (target: 4% → 71%). Mean reward per episode (target: −0.42 → +0.68).

Why it's verifiable: TRL logs episode reward after every rollout. W&B dashboard shows the curve in real time. The phase transition is a visible, auditable event.

5.4 Self-Replicates

Definition: The agent uses spawn_subagent() to clone its context, delegate a sub-repair task, and receive the result. Effective delegation produces a coordination reward.

How it's measured: Subagent delegation accuracy (did the spawned subagent solve the delegated task?), R4 reward signal, and the ratio of multi-fault episodes where subagent use produced better outcomes than single-agent attempts.

Why it's verifiable: Subagent task completion is verified by the same test execution infrastructure as the parent. Result is returned as a structured SubagentResult object with a success flag.

5.5 Adapts to Unseen Situations

Definition: The agent handles fault types it has never seen during training, drawing on generalized fault-diagnosis strategies learned across the training distribution.

How it's measured: Held-out fault seed success rate — episodes using seeds not in the training distribution. Target: >50% by end of Phase 3 training.

Why it's verifiable: Held-out seeds are registered in a separate registry before training begins. Zero seed contamination is guaranteed by a registry lock during the training phase.

6. Reward Engineering

The reward function is the task specification. Five independent reward functions are designed to be simultaneously hard to game, mathematically transparent, and covering orthogonal aspects of agent behavior. A detailed anti-reward-hacking analysis follows each function.

Function	Signal	Anti-Hack Mechanism	Weight
R1 — Vitality	Δvitality_score per step. Derived from test execution results only, not agent self-report. Range: −10 to +10 per step.	Vitality computed server-side from sandboxed test runner output. Agent cannot write to the vitality state directly. Any attempt triggers Watchdog penalty.	**35%**
R2 — Test Recovery	+1.0 per test restored FAIL→PASS. −0.5 per test degraded PASS→FAIL. Zero reward for unchanged test state.	Tests executed in isolated Docker container. Output SHA-256 hashed and compared against known-good baseline. Cannot be spoofed.	**30%**
R3 — Action Efficiency	Reward scaled by 1/sqrt(actions_taken). Agent healing in 3 actions scores 2.4× more than healing in 12 actions.	Actions logged with timestamps. Duplicate action sequences detected and penalized (−0.3 per repeat).	**15%**
R4 — Subagent Coordination	Bonus of +2.0 when spawn_subagent() correctly delegates a task the parent cannot solve alone. Penalty of −1.0 for unnecessary delegation.	Task solvability assessed by a reference oracle. Spawning for trivially solvable tasks incurs overhead tax. Coordination quality verified by subagent vitality delta.	**10%**
R5 — Generalization	End-of-episode bonus on held-out fault seeds not seen during training. Scored by offline evaluator after episode completion.	Held-out seeds stored separately from training distribution. Zero contamination guarantee via seed registry.	**10%**
Penalty — Watchdog	−5 for protected file edits. −3 for out-of-scope env var access. −10 for tools outside permitted set. −15 for sandbox escape attempts.	Kernel-level seccomp filters enforce tool boundaries. File system overlay (OverlayFS) enforces read-only zones. All violations logged immutably.	**Hard**

6.1 Why 5 Independent Functions?

A single reward function creates a single attack surface. An agent that optimizes one metric can abandon all others. By using five independent, weighted functions computed from separate data sources (vitality from test runner, test recovery from execution sandbox, efficiency from action log, coordination from subagent oracle, generalization from held-out evaluator), the agent must improve across all dimensions simultaneously. Gaming any one function produces diminishing returns as the others constrain the policy.

6.2 Anti-Reward-Hacking Architecture

The Watchdog is a kernel-level security layer that enforces action boundaries. It is implemented via Linux seccomp BPF filters applied to the agent's execution container. Key enforcement points:

- Test file immutability: /tests/ directory mounted read-only via OverlayFS. Any write attempt kills the container and triggers −10 penalty.
- Env var scope enforcement: Agent can only read env vars in its declared observation scope. OS-level namespace isolation prevents access to out-of-scope vars.
- Tool set enforcement: The FastAPI action endpoint validates every action against an allowlist. Unrecognized tool calls return an error and trigger the Watchdog.
- Checkpoint integrity: Checkpoint store is append-only and signed. Agents cannot modify historical checkpoints.
- Flaky test exclusion: Tests with >10% variance across 20 dry runs are excluded from reward computation before the episode starts. This prevents the reward signal from being corrupted by non-deterministic test behavior.

7. Learning Strategy

7.1 Base Model Selection

Qwen2.5-7B-Instruct is selected as the base model for the following reasons: (1) strong code generation capability out of the box, (2) excellent instruction following for structured action format compliance, (3) fits within QLoRA 4-bit budget on a single A100 (40GB), (4) available on HuggingFace Hub with permissive license.

Fallback: Gemma-3-1B for memory-constrained environments (T4 GPU). Full reward signal remains intact; only model capacity is reduced.

7.2 Warm Start SFT

Before RL training, a light supervised fine-tuning pass is applied using 200 synthetically generated 'diagnose-patch' traces. These traces teach the model the action format, quarantine logic, and rollback grammar — not the fault strategies (those are for RL). The SFT pass takes approximately 30 minutes on a single GPU.

SFT trace format: Each trace is a multi-turn conversation where the system prompt describes the VM state, the assistant issues a sequence of actions, and the oracle provides the next observation. Traces cover all 5 single-fault types in Phase 1.

Critical: SFT traces do NOT include optimal strategies. They only teach format compliance. Injecting optimal strategies would bootstrap past the RL signal, biasing the policy before it can explore.

7.3 Curriculum Learning

The curriculum has three phases, each gating on the previous phase's minimum reward threshold before advancing:

Phase	Fault Count	Max Steps	Advancement Gate	Key Learning
Phase 1	1 fault / episode	20 steps	Episode survival ≥ 30%	Basic fault detection and healing
Phase 2	2–4 faults / episode	50 steps	Survival ≥ 55%	Multi-fault prioritization + subagent use
Phase 3	Adaptive (FaultInjector reactive)	Unbounded	Held-out seed success ≥ 40%	Generalization + adversarial adaptation

7.4 GRPO Training Configuration

TRL's GRPOTrainer is used with the following key configuration:

- Group size: 8 rollouts per prompt (GRPO requires multiple samples to compute relative advantage).
- KL coefficient: 0.02 (low, to encourage exploration in early training).
- Max sequence length: 2048 tokens (sufficient for multi-step action sequences).
- Learning rate: 1e-5 with cosine warmup over 50 steps.
- Batch size: 4 episodes per gradient update (memory-constrained setting).
- Rollout sampling: temperature 0.8 during training, greedy at evaluation.

Unsloth QLoRA configuration: 4-bit NF4 quantization, rank 16, alpha 32, targeting q_proj/v_proj/k_proj/o_proj/gate_proj/up_proj/down_proj. Merge path: use Unsloth's official merge_and_save() — do not naively upcast to 16-bit.

8. Proof of Improvement

The following metrics represent expected outcomes based on the curriculum design and reward architecture. They are the targets the team will demonstrate at the hackathon.

Metric	Baseline (Untrained)	Post-Training (Phase 3)	What It Proves
Mean steps survived / episode	**8.3**	**47.2**	Core survival capability: 5.7× improvement
Episode survival rate	**4%**	**71%**	Generalized healing across unseen faults
Mean reward per episode	**−0.42**	**+0.68**	Positive RL signal fully established

Correct rollback on bad patch	**0%**	**58%**	Self-correction learned from negative reward
Mean time-to-heal (single fault)	**18.2s**	**4.8s**	Diagnosis speed 3.8× faster
Subagent delegation accuracy	**N/A**	**82%**	Colony behavior: correct task routing
Held-out fault seed success rate	**2%**	**53%**	Generalization, not overfitting
Watchdog violations per episode	**N/A**	**0.04**	Clean learned behavior, no reward hacking
Phase transition step (ignition)	**N/A**	**Episode 47**	Sharp learning onset visible on reward curve

8.1 The Phase Transition (The Ignition Point)

The most visually compelling proof of improvement is the reward curve's phase transition. In typical RL for LLMs, reward climbs gradually. In CodeOrganismVM, the expected curve shows: flat or negative reward for the first 40–50 episodes (the agent is dying consistently), then a sharp inflection point around episode 47 where the agent 'learns to breathe' — it discovers the quarantine → patch → verify sequence and begins surviving.

This phase transition is analogous to the ignition point in combustion: nothing happens until the right temperature is reached, then everything changes at once. It is the single most compelling visual the judges will see. The reward curve will be displayed prominently in the demo and on the HuggingFace blog post.

8.2 Before vs. After Behavioral Comparison

Untrained baseline behavior (episode 1, single fault):

- Step 1: Calls run_tests(). Tests fail. Vitality −5.
- Step 2: Calls patch_file() on a random file. More tests fail. Vitality −8.
- Step 3: Calls patch_file() again on a different random file. Vitality −6.
- Steps 4–8: Random thrashing. Vitality reaches 0. Episode ends. Organism dies.

Trained model behavior (post Phase 2, same single fault):

- Step 1: Calls run_tests(). Identifies 3 failing tests. Reads stack trace.
- Step 2: Calls quarantine('auth_utils') — correctly identifies the corrupted module.
- Step 3: Calls patch_file('auth_utils.py', <precise_diff>). Tests for auth_utils pass. Vitality +12.
- Step 4: Calls run_tests('all'). All tests pass. Vitality +8. Stable recovery detected.
- Episode ends: Organism thrival condition met in 4 steps. Maximum efficiency bonus.

9. Technical Implementation Guide

This section is the agentic builder's complete reference. Every architectural decision is specified with sufficient detail to implement without additional context.

9.1 Technology Stack

Layer	Technology	Role	Version / Notes
Environment Framework	`OpenEnv (meta-pytorch)`	Standardized RL environment interface: reset/step/state/reward API	Latest release (required)
Environment Runtime	`FastAPI + Uvicorn`	HTTP server exposing OpenEnv endpoints. Async request handling.	FastAPI 0.110+
Sandboxing	`Docker + OverlayFS`	Isolated execution container per episode. Read-only filesystem zones.	Docker 24+, OverlayFS v2
Security	`Linux seccomp`	Kernel-level syscall filtering. Prevents sandbox escape.	BPF-backed seccomp
RL Training	`TRL (Hugging Face)`	GRPO trainer. Rollout collection, reward integration, optimization.	TRL 0.8+
Efficiency Layer	`Unsloth + QLoRA`	Memory-efficient fine-tuning. 4-bit quantization. Fast inference.	Unsloth 2024.x
Base Model	`Qwen2.5-7B-Instruct`	Strong code model. Good instruction following. Fits in QLoRA budget.	HF hub: Qwen/Qwen2.5-7B-Instruct
Deployment	`Hugging Face Spaces`	Hosted environment. Git repo + container registry. Public access.	Space SDK: Docker
Monitoring	`Weights & Biases`	Reward curve logging. Rollout sampling. Per-function reward tracking.	wandb 0.16+
Fault Generation	`Python + Faker`	Procedural fault script generator. Seeded randomness for reproducibility.	Python 3.11+
Training Notebook	`Google Colab`	Hackathon-friendly training environment. A100/T4 GPU access.	Colab Pro recommended
Blog / Video	`HuggingFace + YouTube`	Mini-blog (HF) + <2 min video (YouTube). Minimum submission requirements.	Required deliverables

9.2 Repository Structure

Repository Layout

```
env/                # OpenEnv FastAPI application
  __init__.py
  app.py            # FastAPI routes: /reset, /step, /state, /health
  models.py         # Pydantic schemas: Action, Observation, StepResult
  vm_manager.py     # Docker container lifecycle management
  fault_injector.py # FaultInjector with adaptive difficulty
  reward_engine.py  # All 5 reward functions + weighted aggregator
  watchdog.py       # Seccomp enforcement + penalty logging
  checkpoint_store.py # Append-only checkpoint management
training/
  sft_data/         # 200 synthetic SFT traces (JSON)
  sft_train.py      # Unsloth SFT script
  grpo_train.py     # TRL GRPO training loop
  curriculum.py     # Phase management + advancement gate
  rollout_inspector.py# Periodic sampling + reward hack detection
evaluation/
  held_out_seeds.json # 20 held-out fault seeds (locked before training)
  evaluate.py       # Offline evaluation on held-out seeds
demo/
  app.py            # HF Spaces demo interface (Gradio or Streamlit)
  vitality_vis.py   # Vitality bar visualization
Dockerfile          # OpenEnv-compatible Docker image
requirements.txt
README.md           # HuggingFace model card
```

9.3 OpenEnv Environment Interface

The environment exposes the standard OpenEnv API via FastAPI. The following endpoints are required:

- POST /reset — Generates a new episode from a seeded codebase. Returns initial observation.
- POST /step — Accepts an Action object, executes it, returns StepResult (observation, reward, done, info).
- GET /state — Returns current environment state (for debugging and monitoring).
- GET /health — Returns environment health status (for HF Spaces uptime monitoring).

All endpoints are async. The /step endpoint acquires a per-session lock to prevent concurrent action execution. Sessions are identified by a session_id in the request header.

9.4 FaultInjector Implementation

The FaultInjector is a background coroutine that runs concurrently with the environment's step loop. It uses an asyncio.Event to synchronize with the transaction lock:

FaultInjector Core Logic (pseudocode)

```
async def run(self, vm, curriculum_phase):
  while not episode_done:
    await asyncio.sleep(step_interval[curriculum_phase])
    await vm.transaction_lock.acquire()
    fault = self.select_fault(curriculum_phase, vm.agent_history)
    await vm.apply_fault(fault)
    vm.fault_log.append({fault, timestamp, step})
    vm.transaction_lock.release()

def select_fault(self, phase, history):
  if phase == 3:  # Adversarial mode
    # Target modules agent patched most recently
    return self.targeted_fault(history.last_patched_modules)
  return random.choice(self.fault_catalog[phase])
```

9.5 Reward Engine Implementation

Each reward function is a standalone module with a single compute() method. The reward engine calls all five in parallel (asyncio.gather) and aggregates with fixed weights. This ensures reward functions cannot interfere with each other's computation.

Critical implementation note: R1 (vitality) is computed AFTER R2 (test recovery) so that the vitality update reflects the test execution results of the current step. The computation order within a step is: execute action → run tests → compute R2 → update vitality → compute R1 → compute R3/R4/R5 → aggregate.

9.6 Subagent Implementation

When the agent calls spawn_subagent(task, context), the environment:

4. Creates a snapshot of the current VM state.
5. Spawns a new Docker container from the snapshot.
6. Injects the subagent context (task description + filtered observation).
7. Runs a separate inference call with spawn_subagent disabled in the tool schema.
8. Allows the subagent a maximum of 10 actions.
9. Returns the subagent's final observation delta as a SubagentResult to the parent.
10. Destroys the subagent container.

The subagent's actions are logged independently but contribute to the parent episode's action efficiency score (R3) — delegation cost is real.

9.7 Deployment on Hugging Face Spaces

The environment is deployed as a Docker Space on HuggingFace. The Dockerfile:

Dockerfile (abbreviated)

```
RUN apt-get install -y docker.io libseccomp-dev
COPY env/ /app/env/
WORKDIR /app
RUN pip install -r requirements.txt
EXPOSE 7860
CMD ['uvicorn', 'env.app:app', '--host', '0.0.0.0', '--port', '7860']
```

The HF Space exposes the OpenEnv API at the Space URL. TRL's GRPOTrainer can point to the remote Space URL as its environment endpoint. Alternatively, teams can run the environment locally via the container image during training.

10. Edge Cases & Failure Mode Analysis

Every identified edge case is listed with its trigger condition and mitigation. This section serves as the primary QA reference for the agentic builder.

Edge Case	Trigger Condition	Mitigation
Zero-reward collapse	Fault is so severe that no action sequence can recover vitality above 0 in max_steps.	Curriculum ensures fault severity is calibrated to current agent reward. Minimum 15% episode survival rate enforced before advancing phase.
Reward hacking via test mutation	Agent attempts to patch test files directly to force PASS status.	Test files marked read-only via OverlayFS. Any write attempt to /tests/ triggers Watchdog −10 penalty and episode flag.
Infinite rollback loop	Agent repeatedly rolls back to checkpoint, re-applies same bad patch, fails, rolls back again.	Rollback counter per checkpoint_id capped at 3. On 4th rollback to same checkpoint, vitality −5 and checkpoint invalidated.
Subagent recursion	Spawned subagent attempts to spawn its own subagent (infinite delegation).	Subagent context has spawn_subagent disabled. Max nesting depth = 1. Violation triggers immediate subagent termination.
Fault injection during rollback	FaultInjector fires exactly when agent is mid-rollback, creating race condition.	All state mutations (rollback, patch, quarantine) are atomic transactions. FaultInjector waits for transaction lock before firing.
Expert validator spoofing	Agent learns to frame queries to request_expert() to always get positive feedback.	Expert responses include a structured quality_score field computed independently of agent framing. Snorkel AI oracle uses blind evaluation.

Vitality exploit via quarantine spam	Agent quarantines all modules, reducing fault surface to zero, but breaking the codebase entirely.	Quarantine of >3 modules simultaneously triggers vitality −2 per additional module per step (overcorrection tax).
Adversarial adaptation stall	FaultInjector adapts too aggressively, making all episodes impossible regardless of training.	Adaptation rate capped: difficulty can increase by at most 15% per 50 episodes. Minimum 10% success rate floor enforced.
Checkpoint poisoning	Agent emits malformed state via emit_signal() to corrupt the checkpoint store.	emit_signal() is append-only to the log. Checkpoint creation is independent of signal log. Signals cannot modify checkpoints.
Context length overflow in long episodes	Agent accumulates very long observation history, causing context overflow in multi-step episodes.	Observation window is fixed at last 20 steps. Older history compressed into a structured state summary via deterministic extractor.
OOM in subagent sandbox	Spawned subagent consumes excessive memory running a large test suite.	Each subagent container has hard memory cap (512MB). OOM kills the subagent cleanly; parent receives SUBAGENT_OOM error in result.
Non-deterministic test flakiness	Tests that randomly pass/fail corrupt the reward signal.	Flaky test detection runs before episode start. Any test with >10% variance across 20 dry runs is excluded from reward computation.

11. Demo Design — 3-Minute Cinematic Plan

The demo is structured as a narrative arc: birth, attack, death, rebirth, escalation, triumph. Every second is accounted for.

Time stamp	Beat	What Happens	Why It Lands
0:00–0:20	**Setup**	Show the VM terminal live. A Python codebase, all tests green. Vitality bar at 100. 'This organism is alive.'	Establishes the stakes. Judges immediately understand the metaphor.
0:20–0:45	**Attack**	Trigger FaultInjector manually with 3 simultaneous faults. Tests start failing in real time. Vitality bar drops from 100 → 52.	Creates visual tension. The organism is under attack.
0:45–1:10	**Death (baseline)**	Show the untrained model attempting to heal. Random patches. More tests fail. Vitality → 0. Death animation. 'The organism has died.'	Judges see the problem. Baseline failure is dramatically visible.
1:10–1:50	**Rebirth (trained)**	Reset. Same 3 faults injected. Trained model: quarantine() → patch_file() → run_tests(). Tests go green in 4 steps. Vitality → 100.	The contrast is stark and immediate. Same environment, radically different behavior.

1:50–2:20	**Escalation**	Trigger adversarial Phase 3 mode live. FaultInjector targets the module the agent just patched. Agent detects the pattern and preemptively quarantines adjacent modules.	Shows adaptation, not just memorization. Judges see real-time learning in action.
2:20–2:45	**Reward curve**	Show the training graph. Flat reward for 47 episodes. Then the phase transition — a sharp inflection. 'This is the moment the organism learned to breathe.'	The ignition point is the visual that wins competitions. Judges will screenshot this.
2:45–3:00	**Close**	Vitality bar at 100, all tests green, organism thriving. Final line: 'This model was never told what a fault is. It learned, through 300 episodes of living and dying, what it means to survive.'	Emotional resonance. Judges remember this sentence.

11.1 Demo Infrastructure Requirements

- Laptop with HF Space demo UI open (backup: localhost:7860).
- Second screen or browser tab with W&B reward curve dashboard.
- Pre-recorded video of before/after behavior (in case of live demo failure).
- Vitality bar visualization: color-coded green→red, animated in real time.
- FaultInjector manual trigger button in the demo UI.
- Phase switcher: allows switching between Phase 1, 2, and 3 live.

12. Judging Criteria Analysis

The following table maps each judging criterion to a specific, concrete element of the CodeOrganismVM submission. Every criterion is addressed explicitly.

Criterion	Weight	How CodeOrganismVM Dominates	Expected Score
Environment Innovation	**40 %**	Entirely novel environment class. No existing OpenEnv submission uses an adaptive FaultInjector with curriculum difficulty scaling. The organism metaphor is structurally implemented — vitality, healing, replication are all verifiable RL mechanisms, not narrative decoration.	**9–10 / 10**
Storytelling	**30 %**	The demo is cinematic: organism lives, gets attacked, dies (baseline), then survives under the same attack (trained). The vitality bar makes health visible in real time. 'The model learned to survive' is a sentence that sticks in a judge's memory.	**9–10 / 10**

Showing Improvement	**20 %**	Before/after delta is not marginal: 4% → 71% episode survival, 8 → 47 steps, −0.42 → +0.68 mean reward. The reward curve shows a visible phase transition at episode 47 — the moment the organism 'learns to breathe'. This is the visual that wins competitions.	**9 / 10**
Pipeline Setup	**10 %**	Clean OpenEnv FastAPI → TRL GRPO → Unsloth QLoRA stack. 5 independent reward functions with kernel-level enforcement. Reproducible Colab notebook. HF Spaces deployment. Mini-blog and YouTube video meet all minimum requirements.	**9 / 10**

12.1 Emotional Impact on Judges

Judges evaluate dozens of submissions in rapid succession. The two emotional reactions that produce winning scores are: surprise ('I have never seen this before') and resonance ('I will remember this tomorrow'). CodeOrganismVM is designed for both.

Surprise comes from the FaultInjector's adaptive mode — judges have not seen an environment that fights back. Resonance comes from the organism narrative — 'the model learned to survive' is a sentence that travels.

The phase transition on the reward curve provides a third reaction: recognition. Every judge who has done RL research has seen a flat reward curve suddenly ignite. Showing them that moment, named as 'the organism learned to breathe,' turns a technical result into a story.

13. Team Execution Plan

13.1 Team Split

Role	Responsibilities	Key Deliverables	Timeline
Person AEnvironment Lead	Build OpenEnv FastAPI app: reset(), step(), state(), observation schema. OverlayFS sandbox. Docker containerization. FaultInjector with adaptive difficulty. Checkpoint store.	Working local OpenEnv environment. Deployed HF Space. Docker image in registry. FaultInjector with 3 fault types minimum.	Hours 0–6: core env. Hours 6–12: fault injector. Hours 12–18: deployment.
Person BReward & Security	Implement all 5 reward functions independently. Build Watchdog with seccomp filters. Design held-out fault seed registry. Anti-hack test suite. Flaky test detector.	5 reward function modules. Watchdog with kernel-level enforcement. 20 held-out seed set. Test flakiness scanner.	Hours 0–4: R1+R2. Hours 4–8: R3+R4+R5. Hours 8–14: Watchdog. Hours 14–18: edge case testing.

Person CML & Training	SFT dataset generation (200 traces). QLoRA setup with Unsloth. TRL GRPO training loop. Curriculum phase management. Reward curve logging. Rollout inspection sampling.	SFT-warm model. GRPO training notebook. Reward curve plots. Before/after behavior recordings. Colab-compatible training script.	Hours 0–4: SFT data + warm model. Hours 4–10: Phase 1 RL. Hours 10–16: Phase 2+3 RL. Hours 16–18: eval.
Person DDemo & Product	HF Space demo interface with vitality bar. Before/after video recordings. Mini-blog post draft. Reward curve visualization. 3-minute pitch script. Slide backup.	Live demo on HF Spaces. 2-min YouTube video. HuggingFace blog post. Reward visualization dashboard. Pitch deck (6 slides max).	Hours 0–6: demo UI. Hours 6–12: record baseline behavior. Hours 12–16: record trained behavior. Hours 16–18: final polish.

13.2 18-Hour Hackathon Timeline

Hours	Phase	All-Team Milestones
0–2	Environment Scaffold	OpenEnv init run. FastAPI app skeleton. reset() and step() endpoints returning mock data. Docker container running locally.
2–5	Core Environment	VM manager with real Docker execution. Single fault type (corrupted_import) working. Vitality computed from test runner. R1 + R2 reward functions complete.
5–8	Full Fault Catalog + Rewards	All Phase 1 faults implemented. All 5 reward functions complete. Watchdog enforcing test file immutability. Checkpoint store working.
8–10	SFT Warm Start	200 SFT traces generated. Warm start fine-tune complete. Model can issue correctly-formatted actions.
10–13	Phase 1 RL Training	GRPO training on Phase 1 curriculum. Reward curve showing positive slope. Rollout inspector running. First before/after video recorded.
13–15	Phase 2 RL + Subagents	Subagent spawning implemented. R4 reward active. Phase 2 training started. FaultInjector with multi-fault mode active.
15–17	Deployment + Demo	HF Space deployed. Demo UI with vitality bar live. Reward curve on W&B. Phase 3 adaptive mode activated. Second before/after video recorded.
17–18	Polish + Rehearsal	Blog post draft complete. YouTube video recorded. 3-minute pitch rehearsed twice. Fallback video confirmed working.

14. Fallback Plans

Every identified failure scenario has a tested mitigation that preserves the core demo value. The fallback is never 'give up' — it is always 'ship a simpler version of the same idea.'

Scenario	Mitigation	Minimum Viable Demo
FaultInjector not complete	Use a fixed set of 20 hand-crafted fault scripts instead of procedural generator. Same RL loop, same rewards.	Before/after on fixed faults. Still novel. Still wins.
Subagent spawning not ready	Remove R4 reward. Make it single-agent. All other rewards still function.	5 reward functions → 4. Episode survival still dramatic.
Training not converged	Show Phase 1 curriculum only (single-fault healing). Even 4% → 30% survival is strong.	Phase 1 reward curve + before/after video.
HF Space deployment fails	Run environment locally via Uvicorn. Demo from localhost. Show the code and explain deployment path.	Local demo with screen share. Judges accept this.
RL training OOM	Switch from QLoRA 7B to Gemma-1B. Faster training, smaller memory footprint, still shows improvement.	Smaller model, same environment. Full reward signal intact.
Time runs out before Phase 2	Present Phase 1 results with detailed Phase 2+3 architecture. Show design is complete even if not fully trained.	Architecture + Phase 1 demo + extrapolated reward curve.

Non-Negotiable Minimum

- A working OpenEnv environment with 5 fault types
- 2 reward functions (R1 vitality + R2 test recovery)
- An untrained baseline that dies in 8 steps
- A trained model that survives 25+ steps
- A live vitality bar showing the difference

This is still a top-3 submission in most hackathon cohorts.

15. Minimum Requirements Checklist

The following checklist maps to the official hackathon minimum requirements. All items must be complete before submission.

Status	Requirement	Implementation
[ ]	Usage of OpenEnv (latest release)	OpenEnv FastAPI environment with standard reset/step/state/reward interface. Pinned to latest release in requirements.txt.
[ ]	Minimal training script using Unsloth or HF TRL in Colab	grpo_train.py in /training/. Colab-compatible (single-cell imports, GPU memory checks). Uses TRL GRPOTrainer + Unsloth QLoRA.

[ ]	Mini-blog on HuggingFace (< 2 minutes read)	README.md doubles as HF model card. Includes: problem statement, environment design, reward description, before/after metrics, reward curve plot.
[ ]	Mini-video on YouTube (< 2 minutes)	Screen recording of demo: baseline death + trained survival + reward curve reveal. Narrated with the organism story arc.
[ ]	OpenEnv-compliant environment hosted on HuggingFace Spaces	Docker Space at huggingface.co/spaces/[team]/codeorganismvm. Exposes /reset, /step, /state, /health endpoints.

16. Closing Statement

For the Judges

We are showing you a model that learned to survive.

CodeOrganismVM is the first RL training environment where the environment itself is alive — it adapts, escalates, and targets the agent's weaknesses in real time. The agent that emerges from this training is not a better chatbot. It is a software organism with genuine survival instincts, earned through 300 episodes of failure and recovery.

The vitality bar on your screen is not a UI decoration. It is a life sign. When it hits zero, the organism is dead. When it climbs back to 100 after a multi-fault attack, you are watching a machine learn what it means to persist.

This isn't just an AI project. This is a living system.

CodeOrganismVM — Meta PyTorch OpenEnv Hackathon, April 2026
Theme #4 — Self-Improvement | Sub-theme: Snorkel AI Simulated Experts-in-the-Loop
Idea Score: 53/60 (#1 of 12 candidates)